Name: bh. Sumitranand sharma

MERGE SORT:

CODE:

```
GNU nano 7.2
#include <stdio.h>

#define N 12

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
int main() {
    int roll[N] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};

    quickSort(roll, 0, N - 1);

    printf("Sorted Roll Numbers using Quick Sort:\n");
    for (int i = 0; i < N; i++)
        printf("%d ", roll[i]);

    return 0;
}
```

OUTPUT:

```
achyuth@achyuth-VirtualBox:~$ nano quick.c
achyuth@achyuth-VirtualBox:~$ gcc quick.c -o quick
achyuth@achyuth-VirtualBox:~$ ./quick
Sorted Roll Numbers using Quick Sort:
110 111 112 117 122 123 133 141 147 149 151 157 achyuth@achyuth-VirtualBox:~$
```

Complexity Analysis of Quick Sort

- **Best Case: O(n log n),**
when the pivot divides the array into two nearly equal halves.

- **Average Case: O(n log n),**
when the elements are in random order and partitions are balanced.

- **Worst Case: $O(n^2)$,**
when the array is already sorted or reverse sorted and the pivot is chosen poorly.

QUICK SORT:

CODE:

```
  GNU nano 7.2
#include <stdio.h>

#define N 12

void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}
```

```c
int main() {
    int roll[N] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};

    mergeSort(roll, 0, N - 1);

    printf("Sorted Roll Numbers using Merge Sort:\n");
    for (int i = 0; i < N; i++)
        printf("%d ", roll[i]);

    return 0;
}
```

OUTPUT:

```
achyuth@achyuth-VirtualBox:~$ nano merge.c
achyuth@achyuth-VirtualBox:~$ gcc merge.c -o merge
achyuth@achyuth-VirtualBox:~$ ./merge
Sorted Roll Numbers using Merge Sort:
110 111 112 117 122 123 133 141 147 149 151 157 achyuth@achyuth-VirtualBox:~$
```

Complexity Analysis of Merge Sort

- **Best Case: O(n log n),**
when the array is already sorted or nearly sorted.

- **Average Case: O(n log n),**
when the array elements are in random order.

- **Worst Case: O(n log n),**
when the array is sorted in reverse order.